

ARARE AI Manus P3 Prompt

Mobile sharing PDF. Attach the project ZIP separately in Manus.

添付ZIPを展開し、ARARE AI の開発を引き継いでください。

このManusアカウントには過去の会話履歴がない前提です。

過去チャットには依存せず、添付ZIP内のコードとドキュメントを唯一の正として扱ってください。

まず必ず読んでください。

- README.md
- docs/HANDOFF.md
- docs/STATUS_REPORT.md
- docs/TODO_NEXT.md
- docs/AUTH_DESIGN.md
- docs/CODEX_VERIFICATION_P1D.md
- docs/CREDIT_SAFE_HANDOFF.md
- prisma/schema.prisma
- apps/api/src/lib/webhookSignature.ts
- apps/api/src/routes/index.ts
- apps/api/src/notifications/notificationDispatcher.ts
- apps/api/src/services/twilioWebhookService.ts
- apps/api/src/services/openaiResponderAdapter.ts
- apps/api/src/services/lineShiftParserAdapter.ts

プロジェクト概要:

ARARE AI は、メンズエステ店舗向けの AI受付・予約自動化SaaSです。

将来的に100店舗運用を想定しています。

電話AI、LINE、Webチャットから問い合わせを受け、店舗ナレッジに基づいて回答し、仮予約を作成します。

予約確定は必ず店舗スタッフ承認後です。

絶対に守るルール:

- AIが予約を直接 CONFIRMED にする実装は禁止
- 予約確定は ReservationHold -> staff approval -> Reservation(CONFIRMED) のみ
- 100店舗マルチテナント前提で storeId 分離を維持
- 本番キーをコード・docs・ログ・ZIPに含めない
- .env は納品ZIPに含めない
- 外部APIは環境変数が dummy / 未設定なら必ず mock fallback
- Twilio / LINE / OpenAI / Clerk の実キーを使う実送信・実応答は今回まだ行わない
- クレジットが少なくなったら docs/CREDIT_SAFE_HANDOFF.md に従って即ZIP納品

今回のゴール:

P3 本番接続準備・堅牢化を進めてください。

本番キー投入前に、事故らない構造・厳密E2E・署名検証・Adapter切替・ヘルスチェックを整備します。

今回やること:

1. Webhook署名検証

- Twilio署名検証を公式SDK方式に寄せる
- X-Twilio-Signature を検証
- form-urlencoded / JSON の両方を考慮
- WEBHOOK MOCK_MODE=true または TWILIO_AUTH_TOKEN=dummy の場合のみ mock 通過
- WEBHOOK MOCK_MODE=false かつ署名なし/不正署名は 403
- LINEは x-line-signature を LINE_CHANNEL_SECRET で HMAC-SHA256 + base64 検証
- LINEも dummy / mock 時のみ通過、不正署名は403

2. 厳密E2E化

- DB未接続による500をPASS扱いしない
- e2e.auth.ts / e2e.notification.ts / e2e.webhook.ts / e2e.shiftParser.ts を修正
- 期待ステータスを厳密化
- 500は必ずFAIL
- NotificationLog / CallLog / AIConversationLog / Message / LineShiftParseJob の保存を実DBで検証
- dedupe / retry / status遷移 / storeId分離を実DBで確認

3. 通知基盤の本番切替準備

- twilioSmsAdapter.ts を強化
- dummy/未設定ならmock送信
- 本番値がある場合のみTwilio SDK送信へ切替
- 送信結果SID、status、errorCode、errorMessageを NotificationLog に反映
- LINE Messaging Adapterも同様に、dummy時mock、本番値時のみ送信へ切替
- LINE userId未登録時のfallbackを明確化

4. AI/電話安全性

- OpenAIキー未設定/dummyならmock
- OpenAI本番接続用の差し替え口は作るが、今回実送信は不要
- AI応答から直接 Reservation(CONFIRMED) を作れないことをテスト
- Voice Webhook / Conversation Relay 経由でも ReservationHold まで
- AIが高信頼でもスタッフ承認待ちにする
- ナレッジ外は「確認が必要です。店舗に確認して折り返します。」を維持
- LLM出力のJSON parse失敗時fallbackを実装

5. LINEシフト解析の運用化

- confidenceScore < 0.5 は PENDING_REVIEW のまま
- 高信頼でも即Shift反映ではなく、店舗設定で承認必須にできる構造にする
- 解析結果一覧/確認APIを整備
- 誤解析時の修正・却下APIを追加できるなら追加
- 低信頼がShiftに直接入らないことをE2Eで厳密に検証

6. ヘルスチェック

- /health
- /ready
- /stores/:storeId/health

を整備してください。

確認項目:

- DB接続
- Notification adapter状態
- Voice adapter状態
- AI adapter状態
- LINE adapter状態
- Clerk mock/real mode
- 店舗別READY/NOT_READY理由

7. ドキュメント更新

必ず更新:

- README.md
- docs/HANDOFF.md
- docs/STATUS_REPORT.md
- docs/TODO_NEXT.md
- docs/AUTH_DESIGN.md
- docs/CODEX_VERIFICATION_P1D.md

- docs/CREDIT_SAFE_HANDOFF.md

追加推奨:

- docs/PRODUCTION_READINESS.md
- docs/WEBHOOK_SECURITY.md
- docs/ENVIRONMENT_VARIABLES.md

必須検証:

- pnpm install
- pnpm exec prisma generate --schema=prisma/schema.prisma
- pnpm exec prisma validate --schema=prisma/schema.prisma
- pnpm -r typecheck
- pnpm -r build

実DBで可能なら:

- pnpm exec prisma migrate dev --schema=prisma/schema.prisma
- pnpm exec tsx prisma/seed/seed.ts
- pnpm exec tsx apps/api/test/e2e.reservationHold.ts
- pnpm exec tsx apps/api/test/e2e.http.ts
- pnpm exec tsx apps/api/test/e2e.crud.ts
- pnpm exec tsx apps/api/test/e2e.auth.ts
- pnpm exec tsx apps/api/test/e2e.notification.ts
- pnpm exec tsx apps/api/test/e2e.webhook.ts
- pnpm exec tsx apps/api/test/e2e.shiftParser.ts

可能ならNext.jsを起動し主要ページHTTP 200確認。

納品:

ZIPから必ず除外してください。

- node_modules/
- .next/
- dist/
- .env
- *.log
- *.tsbuildinfo
- 一時ファイル

完了報告は以下だけで短く:

- 実装済み
- 検証済み
- 修正ファイル
- 未実装
- 未検証
- 既知のリスク
- 納品URL
- 次にやること